Patient Risk Assessment System for Non-Communicable Diseases

Project Report | ENSF 608 — Group 6

Submission Date: 7 Dec 2026
Course: ENSF 608: Database Systems
Group: Group 6

Team Members

Student Name	Student ID	Email
Navjot Singh Shienh	30332849	navjot.shienh@ucalgary.ca
Iesrat Jahan Popy	30304908	iesrat.jahanpopy@ucalgary.ca
Nikita Williams	30143759	nikita.williams@ucalgary.ca
Md Jubayer Khan	30336019	mdjubayer.khan@ucalgary.ca

---

1. Introduction & System Overview

1.1 System Purpose

The Patient Risk Assessment System for Non-Communicable Diseases (NCDs) is designed to streamline the tracking, evaluation, and clinical management of patients at risk for chronic non-communicable conditions (such as cardiovascular diseases, diabetes, and chronic respiratory disorders). The centralized database enables healthcare providers to monitor patient vitals, historical diagnostic data, medication compliance, lab test thresholds, and automated risk scoring models to schedule follow-up care effectively.

1.2 UN SDG 3 Alignment

This project directly aligns with **United Nations Sustainable Development Goal 3: Good Health and Well-Being**, specifically:

- **Target 3.4:** By 2030, reduce by one-third premature mortality from non-communicable diseases through prevention and treatment and promote mental health and well-being.
- **Target 3.8:** Achieve universal health coverage, including financial risk protection, access to quality essential health-care services, and access to safe, effective, quality, and affordable essential medicines and vaccines for all.

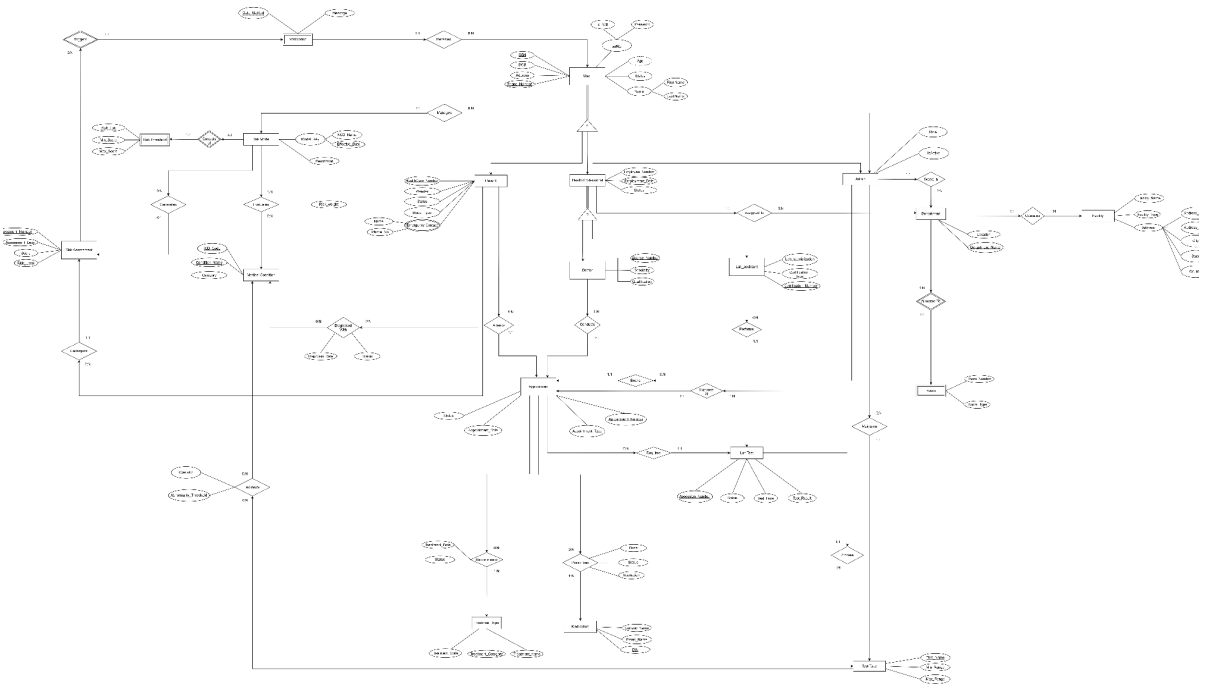
By automating early risk detection, threshold alerts, and improving follow-up tracking for vulnerable patient populations, the system supports proactive healthcare intervention and equitable service distribution.

---

2. Conceptual Database Design

2.1 Entity-Relationship (ER) Diagram

Insert ER Diagram image or vector graphic below.



2.2 Entity Descriptions

- **User Hierarchy (USER, PATIENT, PATIENT_EMERGENCY_CONTACT, HEALTH_PROFESSIONAL, DOCTOR, LAB_ASSISTANT, ADMIN):**
    - **USER:** Base entity for all platform users storing baseline demographic data, hashed password credentials, contact information, and account status.

    - **PATIENT:** Specialized subtype storing provincial healthcare number, gender, blood type, and clinical status.
    - **PATIENT_EMERGENCY_CONTACT:** Emergency contacts associated with a patient.
    - **HEALTH_PROFESSIONAL:** Base subtype for staff storing employee numbers, employment dates, and department assignments.
    - **DOCTOR:** Specialized health professional subtype storing medical license numbers, clinical specialties, and qualifications.
    - **LAB_ASSISTANT:** Specialized health professional subtype storing lab certifications and specializations.
    - **ADMIN:** System administrator subtype managing role permissions and department assignments.
- **Organization (FACILITY, DEPARTMENT, ROOM):**
    - **FACILITY:** Represents healthcare centers, clinics, and hospitals.
    - **DEPARTMENT:** Specific operational units within a facility.
    - **ROOM:** Weak entity representing specific rooms within a department.
- **Appointments, Treatment & Medication (APPOINTMENT, TREATMENT_TYPE, APPOINTMENT_TREATMENT, MEDICATION, APPOINTMENT_PRESCRIPTION):**
    - **APPOINTMENT:** Clinical encounters scheduled between a patient, doctor, and admin in a specific department room.
    - **TREATMENT_TYPE:** Catalog of standard medical treatments and procedures.
    - **APPOINTMENT_TREATMENT:** Treatments administered or recommended during an appointment.
    - **MEDICATION:** Drug identification catalog (DIN, generic name, brand name).
    - **APPOINTMENT_PRESCRIPTION:** Prescriptions generated during an appointment.
- **Lab Testing (TEST_TYPE, LAB_TEST):**
    - **TEST_TYPE:** Master list of lab test types and valid reference ranges maintained by admins.
    - **LAB_TEST:** Laboratory test requests and diagnostic results linked to appointments and lab assistants.
- **Clinical Conditions (MEDICAL_CONDITION, PATIENT_DIAGNOSIS, TEST_THRESHOLD_RULE):**
    - **MEDICAL_CONDITION:** ICD-coded catalog of non-communicable and chronic health conditions.
    - **PATIENT_DIAGNOSIS:** Clinical diagnoses assigned to patients.
    - **TEST_THRESHOLD_RULE:** Rules linking test biomarker ranges to medical conditions for abnormality flags.
- **Risk Modelling (RISK_MODEL, RISK_THRESHOLD, RISK_MODEL_EVALUATION, RISK_ASSESSMENT, NOTIFICATION):**

- **RISK_MODEL:** Versioned non-communicable disease risk computation models managed by admins.
- **RISK_THRESHOLD:** Weak entity defining score ranges for risk levels (Low, Medium, High).
- **RISK_MODEL_EVALUATION:** Weights assigned to medical conditions within a risk model.
- **RISK_ASSESSMENT:** Calculated risk evaluation for a patient.
- **NOTIFICATION:** Automated alerts sent to users regarding risk assessment outcomes.

---

3. Logical Database Model & Normalization

3.1 Logical Database Model Overview

The logical database model translates the conceptual entity-relationship architecture into a fully structured 25-table relational schema. It enforces strict primary and foreign key constraints, composite keys for weak entities and associative tables, unique constraints on critical business attributes (such as email, phone number, healthcare number, employee number, license number, certification number, and condition name), and precise nullability rules derived from structural participation constraints (such as 1:1 mandatory appointment participants and 0:1 optional risk model generation).

3.2 Relational Schema

Notation: underlined = PK, *italic* = FK (#). Composite keys are listed in parentheses. Derived from the Conceptual and Logical models.

User Hierarchy

- **USER** (SSN, FIRST_NAME, LAST_NAME, DOB, ADDRESS, EMAIL, PASSWORD, PHONE_NUMBER, STATUS)
  - Email is UNIQUE. Phone_Number is UNIQUE.
- **PATIENT** (*SSN#*, HEALTHCARE_NUMBER, GENDER, BLOOD_TYPE, STATUS)
  - SSN# → USER(SSN). Healthcare_Number is UNIQUE.
- **PATIENT_EMERGENCY_CONTACT** (*PATIENT_SSN#*, CONTACT_NAME, CONTACT_PHONE_NUMBER)
  - PATIENT_SSN# → PATIENT(SSN)
- **HEALTH_PROFESSIONAL** (*SSN#*, EMPLOYEE_NUMBER, EMPLOYMENT_DATE, STATUS, *DEPARTMENT_NAME#*)
  - SSN# → USER(SSN). Employee_Number is UNIQUE. Department_Name# → DEPARTMENT(Department_Name)

- **DOCTOR** (*SSN#*, LICENSE_NUMBER, SPECIALITY, QUALIFICATION)
    - SSN# → HEALTH_PROFESSIONAL(SSN). License_Number is UNIQUE.
- **LAB_ASSISTANT** (*SSN#*, CERTIFICATION_NUMBER, CERTIFICATION_TYPE, LAB_SPECIALIZATION)
    - SSN# → HEALTH_PROFESSIONAL(SSN). Certification_Number is UNIQUE.
- **ADMIN** (*SSN#*, ROLE, ISACTIVE, *DEPARTMENT_NAME#*)
    - SSN# → USER(SSN). Department_Name# → DEPARTMENT(Department_Name)

Organization

- **FACILITY** (FACILITY_NAME, FACILITY_TYPE, ADDRESS_LINE1, ADDRESS_LINE2, CITY, STATE, COUNTRY)
- **DEPARTMENT** (DEPARTMENT_NAME, LOCATION, *FACILITY_NAME#*)
    - Facility_Name# → FACILITY(Facility_Name)
- **ROOM** (*DEPARTMENT_NAME#*, ROOM_NUMBER, ROOM_TYPE)
    - Department_Name# → DEPARTMENT(Department_Name)

Appointments, Treatment & Medication

- **APPOINTMENT** (APPOINTMENT_NUMBER, APPOINTMENT_DATE, APPOINTMENT_TYPE, STATUS, *PATIENT_SSN#*, *DOCTOR_SSN#*, *ADMIN_SSN#*, *DEPARTMENT_NAME#*, *ROOM_NUMBER#*)
    - Patient_SSN#, Doctor_SSN#, Admin_SSN#, (Department_Name#, Room_Number#) are NOT NULL.
- **TREATMENT_TYPE** (TREATMENT_CODE, TREATMENT_NAME, TREATMENT_CATEGORY)
- **APPOINTMENT_TREATMENT** (*APPOINTMENT_NUMBER#*, *TREATMENT_CODE#*, TREATMENT_DATE, STATUS)
- **MEDICATION** (DIN, GENERIC_NAME, BRAND_NAME)
- **APPOINTMENT_PRESCRIPTION** (*APPOINTMENT_NUMBER#*, *DIN#*, DOSE, INSTRUCTION, STATUS)

Lab Testing

- **TEST_TYPE** (TEST_NAME, MIN_RANGE, MAX_RANGE, *ADMIN_SSN#*)
- **LAB_TEST** (ACCESSION_NUMBER, TEST_DATE, TEST_RESULT, STATUS, *APPOINTMENT_NUMBER#*, *LAB_ASSISTANT_SSN#*, *TEST_NAME#*)

Clinical Conditions

- **MEDICAL_CONDITION** (ICD_CODE, CONDITION_NAME, CATEGORY)
    - Condition_Name is UNIQUE.
- **PATIENT_DIAGNOSIS** (*PATIENT_SSN#*, *ICD_CODE#*, DIAGNOSIS_DATE, STATUS)

- **TEST_THRESHOLD_RULE** (*TEST_NAME#*, *ICD_CODE#*, ABNORMALITY_THRESHOLD, OPERATOR)

Risk Modelling

- **RISK_MODEL** (NCD_NAME, EFFECTIVE_DATE, DESCRIPTION, *ADMIN_SSN#*)
- **RISK_THRESHOLD** (*NCD_NAME#*, *EFFECTIVE_DATE#*, RISK_TIER, MIN_SCORE, MAX_SCORE)
- **RISK_MODEL_EVALUATION** (*NCD_NAME#*, *EFFECTIVE_DATE#*, *ICD_CODE#*, RISK_WEIGHT)
- **RISK_ASSESSMENT** (ASSESSMENT_NUMBER, ASSESSMENT_DATE, SCORE, RISK_LEVEL, *PATIENT_SSN#*, *NCD_NAME#*, *EFFECTIVE_DATE#*)
    - NCD_Name# and Effective_Date# are Nullable (0:1 Generates relationship).
- **NOTIFICATION** (*ASSESSMENT_NUMBER#*, DATE_NOTIFIED, *RECIPIENT_USER_SSN#*, MESSAGE)

3.3 Normalization Proofs (1NF, 2NF, 3NF)

First Normal Form (1NF)

- All attributes contain atomic values (no multivalued attributes or repeating groups).
- Every relation possesses a clearly defined primary key (single or composite).

Second Normal Form (2NF)

- Meets all 1NF criteria.
- All non-key attributes are fully functionally dependent on the entire primary key. In relations with composite primary keys (e.g., `ROOM`, `APPOINTMENT_TREATMENT`, `PATIENT_DIAGNOSIS`, `RISK_THRESHOLD`), no non-key attribute depends on a proper subset of the primary key.

Third Normal Form (3NF)

- Meets all 2NF criteria.
- No transitive dependencies exist; every non-key attribute depends directly and solely on the primary key.

---

4. Physical Implementation & Data Population

4.1 DDL Scripts

```sql
-- DDL Script for Patient Risk Assessment System (NCD Triage System)

CREATE TABLE FACILITY (
    Facility_Name VARCHAR(100) PRIMARY KEY,
    Facility_Type VARCHAR(50) NOT NULL,
    Address_Line1 VARCHAR(100) NOT NULL,
    Address_Line2 VARCHAR(100),
    City VARCHAR(50) NOT NULL,
    State VARCHAR(50) NOT NULL,
    Country VARCHAR(50) NOT NULL
);

CREATE TABLE DEPARTMENT (
    Department_Name VARCHAR(100) PRIMARY KEY,
    Location VARCHAR(100),
    Facility_Name VARCHAR(100) NOT NULL,
    FOREIGN KEY (Facility_Name) REFERENCES FACILITY(Facility_Name)
);

CREATE TABLE ROOM (
    Department_Name VARCHAR(100),
    Room_Number VARCHAR(10),
    Room_Type VARCHAR(50),
    PRIMARY KEY (Department_Name, Room_Number),
    FOREIGN KEY (Department_Name) REFERENCES DEPARTMENT(Department_Name)
);

CREATE TABLE USER (
    SSN CHAR(9) PRIMARY KEY,
    First_Name VARCHAR(50) NOT NULL,
    Last_Name VARCHAR(50) NOT NULL,
    DOB DATE NOT NULL,
    Address VARCHAR(150),
    Email VARCHAR(100) UNIQUE NOT NULL,
    Password VARCHAR(255) NOT NULL,
    Phone_Number VARCHAR(15) UNIQUE,
    Status VARCHAR(20) NOT NULL CHECK (Status IN ('Active', 'Inactive',
'Suspended'))
);

CREATE TABLE PATIENT (
    SSN CHAR(9) PRIMARY KEY,
    Healthcare_Number VARCHAR(20) UNIQUE NOT NULL,
    Gender VARCHAR(20),
```

```
    Blood_Type VARCHAR(3) CHECK (Blood_Type IN ('A+', 'A-', 'B+', 'B-', 'AB+',
'AB-', 'O+', 'O-')),
    Status VARCHAR(20) NOT NULL CHECK (Status IN ('Active', 'Inactive',
'Deceased')),
    FOREIGN KEY (SSN) REFERENCES USER(SSN)
);

CREATE TABLE PATIENT_EMERGENCY_CONTACT (
    Patient_SSN CHAR(9),
    Contact_Name VARCHAR(100),
    Contact_Phone_Number VARCHAR(15),
    PRIMARY KEY (Patient_SSN, Contact_Name, Contact_Phone_Number),
    FOREIGN KEY (Patient_SSN) REFERENCES PATIENT(SSN)
);

CREATE TABLE HEALTH_PROFESSIONAL (
    SSN CHAR(9) PRIMARY KEY,
    Employee_Number VARCHAR(20) UNIQUE NOT NULL,
    Employment_Date DATE NOT NULL,
    Status VARCHAR(20) NOT NULL CHECK (Status IN ('Active', 'On Leave',
'Terminated')),
    Department_Name VARCHAR(100) NOT NULL,
    FOREIGN KEY (SSN) REFERENCES USER(SSN),
    FOREIGN KEY (Department_Name) REFERENCES DEPARTMENT(Department_Name)
);

CREATE TABLE DOCTOR (
    SSN CHAR(9) PRIMARY KEY,
    License_Number VARCHAR(30) UNIQUE NOT NULL,
    Speciality VARCHAR(100),
    Qualification VARCHAR(150),
    FOREIGN KEY (SSN) REFERENCES HEALTH_PROFESSIONAL(SSN)
);

CREATE TABLE LAB_ASSISTANT (
    SSN CHAR(9) PRIMARY KEY,
    Certification_Number VARCHAR(30) UNIQUE NOT NULL,
    Certification_Type VARCHAR(100),
    Lab_Specialization VARCHAR(100),
    FOREIGN KEY (SSN) REFERENCES HEALTH_PROFESSIONAL(SSN)
);

CREATE TABLE ADMIN (
    SSN CHAR(9) PRIMARY KEY,
```

```
    Role VARCHAR(50) NOT NULL,
    IsActive BOOLEAN NOT NULL DEFAULT TRUE,
    Department_Name VARCHAR(100) NOT NULL,
    FOREIGN KEY (SSN) REFERENCES USER(SSN),
    FOREIGN KEY (Department_Name) REFERENCES DEPARTMENT(Department_Name)
);

CREATE TABLE APPOINTMENT (
    Appointment_Number INT PRIMARY KEY AUTO_INCREMENT,
    Appointment_Date DATETIME NOT NULL,
    Appointment_Type VARCHAR(50) NOT NULL,
    Status VARCHAR(20) NOT NULL CHECK (Status IN ('Scheduled', 'Completed',
'Cancelled', 'No-Show')),
    Patient_SSN CHAR(9) NOT NULL,
    Doctor_SSN CHAR(9) NOT NULL,
    Admin_SSN CHAR(9) NOT NULL,
    Department_Name VARCHAR(100) NOT NULL,
    Room_Number VARCHAR(10) NOT NULL,
    FOREIGN KEY (Patient_SSN) REFERENCES PATIENT(SSN),
    FOREIGN KEY (Doctor_SSN) REFERENCES DOCTOR(SSN),
    FOREIGN KEY (Admin_SSN) REFERENCES ADMIN(SSN),
    FOREIGN KEY (Department_Name, Room_Number) REFERENCES ROOM(Department_Name,
Room_Number)
);

CREATE TABLE TREATMENT_TYPE (
    Treatment_Code VARCHAR(20) PRIMARY KEY,
    Treatment_Name VARCHAR(100) NOT NULL,
    Treatment_Category VARCHAR(50)
);

CREATE TABLE APPOINTMENT_TREATMENT (
    Appointment_Number INT,
    Treatment_Code VARCHAR(20),
    Treatment_Date DATE NOT NULL,
    Status VARCHAR(20) NOT NULL CHECK (Status IN ('Recommended', 'In Progress',
'Completed', 'Cancelled')),
    PRIMARY KEY (Appointment_Number, Treatment_Code),
    FOREIGN KEY (Appointment_Number) REFERENCES
APPOINTMENT(Appointment_Number),
    FOREIGN KEY (Treatment_Code) REFERENCES TREATMENT_TYPE(Treatment_Code)
);

CREATE TABLE MEDICATION (
```

```
    DIN CHAR(8) PRIMARY KEY,
    Generic_Name VARCHAR(100) NOT NULL,
    Brand_Name VARCHAR(100)
);

CREATE TABLE APPOINTMENT_PRESCRIPTION (
    Appointment_Number INT,
    DIN CHAR(8),
    Dose VARCHAR(50) NOT NULL,
    Instruction VARCHAR(255),
    Status VARCHAR(20) NOT NULL CHECK (Status IN ('Active', 'Completed',
'Discontinued')),
    PRIMARY KEY (Appointment_Number, DIN),
    FOREIGN KEY (Appointment_Number) REFERENCES
APPOINTMENT(Appointment_Number),
    FOREIGN KEY (DIN) REFERENCES MEDICATION(DIN)
);

CREATE TABLE TEST_TYPE (
    Test_Name VARCHAR(100) PRIMARY KEY,
    Min_Range DECIMAL(10,2),
    Max_Range DECIMAL(10,2),
    Admin_SSN CHAR(9) NOT NULL,
    FOREIGN KEY (Admin_SSN) REFERENCES ADMIN(SSN)
);

CREATE TABLE LAB_TEST (
    Accession_Number VARCHAR(20) PRIMARY KEY,
    Test_Date DATE NOT NULL,
    Test_Result DECIMAL(10,2),
    Status VARCHAR(20) NOT NULL CHECK (Status IN ('Ordered', 'In Progress',
'Completed', 'Cancelled')),
    Appointment_Number INT NOT NULL,
    Lab_Assistant_SSN CHAR(9) NOT NULL,
    Test_Name VARCHAR(100) NOT NULL,
    FOREIGN KEY (Appointment_Number) REFERENCES
APPOINTMENT(Appointment_Number),
    FOREIGN KEY (Lab_Assistant_SSN) REFERENCES LAB_ASSISTANT(SSN),
    FOREIGN KEY (Test_Name) REFERENCES TEST_TYPE(Test_Name)
);

CREATE TABLE MEDICAL_CONDITION (
    ICD_Code VARCHAR(10) PRIMARY KEY,
    Condition_Name VARCHAR(150) UNIQUE NOT NULL,
```

```
    Category VARCHAR(50)
);

CREATE TABLE PATIENT_DIAGNOSIS (
    Patient_SSN CHAR(9),
    ICD_Code VARCHAR(10),
    Diagnosis_Date DATE NOT NULL,
    Status VARCHAR(20) NOT NULL CHECK (Status IN ('Active', 'Resolved',
'Chronic')),
    PRIMARY KEY (Patient_SSN, ICD_Code),
    FOREIGN KEY (Patient_SSN) REFERENCES PATIENT(SSN),
    FOREIGN KEY (ICD_Code) REFERENCES MEDICAL_CONDITION(ICD_Code)
);

CREATE TABLE TEST_THRESHOLD_RULE (
    Test_Name VARCHAR(100),
    ICD_Code VARCHAR(10),
    Abnormality_Threshold DECIMAL(10,2) NOT NULL,
    Operator VARCHAR(2) NOT NULL CHECK (Operator IN ('<', '<=', '>', '>=',
'=')),
    PRIMARY KEY (Test_Name, ICD_Code),
    FOREIGN KEY (Test_Name) REFERENCES TEST_TYPE(Test_Name),
    FOREIGN KEY (ICD_Code) REFERENCES MEDICAL_CONDITION(ICD_Code)
);

CREATE TABLE RISK_MODEL (
    NCD_Name VARCHAR(100),
    Effective_Date DATE,
    Description VARCHAR(255),
    Admin_SSN CHAR(9) NOT NULL,
    PRIMARY KEY (NCD_Name, Effective_Date),
    FOREIGN KEY (Admin_SSN) REFERENCES ADMIN(SSN)
);

CREATE TABLE RISK_THRESHOLD (
    NCD_Name VARCHAR(100),
    Effective_Date DATE,
    Risk_Tier VARCHAR(10) CHECK (Risk_Tier IN ('Low', 'Medium', 'High')),
    Min_Score DECIMAL(5,2),
    Max_Score DECIMAL(5,2),
    PRIMARY KEY (NCD_Name, Effective_Date, Risk_Tier),
    FOREIGN KEY (NCD_Name, Effective_Date) REFERENCES RISK_MODEL(NCD_Name,
Effective_Date)
);
```

```sql
CREATE TABLE RISK_MODEL_EVALUATION (
    NCD_Name VARCHAR(100),
    Effective_Date DATE,
    ICD_Code VARCHAR(10),
    Risk_Weight DECIMAL(5,2) NOT NULL,
    PRIMARY KEY (NCD_Name, Effective_Date, ICD_Code),
    FOREIGN KEY (NCD_Name, Effective_Date) REFERENCES RISK_MODEL(NCD_Name,
Effective_Date),
    FOREIGN KEY (ICD_Code) REFERENCES MEDICAL_CONDITION(ICD_Code)
);

CREATE TABLE RISK_ASSESSMENT (
    Assessment_Number INT PRIMARY KEY AUTO_INCREMENT,
    Assessment_Date DATETIME NOT NULL,
    Score DECIMAL(5,2) NOT NULL,
    Risk_Level VARCHAR(10) NOT NULL CHECK (Risk_Level IN ('Low', 'Medium',
'High')),
    Patient_SSN CHAR(9) NOT NULL,
    NCD_Name VARCHAR(100),
    Effective_Date DATE,
    FOREIGN KEY (Patient_SSN) REFERENCES PATIENT(SSN),
    FOREIGN KEY (NCD_Name, Effective_Date) REFERENCES RISK_MODEL(NCD_Name,
Effective_Date)
);

CREATE TABLE NOTIFICATION (
    Assessment_Number INT,
    Date_Notified DATETIME,
    Recipient_User_SSN CHAR(9) NOT NULL,
    Message VARCHAR(500) NOT NULL,
    PRIMARY KEY (Assessment_Number, Date_Notified),
    FOREIGN KEY (Assessment_Number) REFERENCES
RISK_ASSESSMENT(Assessment_Number),
    FOREIGN KEY (Recipient_User_SSN) REFERENCES USER(SSN)
);
```

---

5. Database Use & Advanced SQL Queries

5.1 Query 1: Risk Level Breakdown

Provides a summary count and percentage distribution of patients across defined risk categories.

```
SQL
SELECT
    Risk_Level,
    COUNT(DISTINCT Patient_SSN) AS TotalPatients,
    ROUND(COUNT(DISTINCT Patient_SSN) * 100.0 / (SELECT COUNT(DISTINCT
Patient_SSN) FROM RISK_ASSESSMENT), 2) AS Percentage
FROM RISK_ASSESSMENT
GROUP BY Risk_Level
ORDER BY
    CASE Risk_Level
        WHEN 'High' THEN 1
        WHEN 'Medium' THEN 2
        WHEN 'Low' THEN 3
        ELSE 4
    END;
```

5.2 Query 2: Unmonitored High-Risk Patients

Identifies patients flagged as High risk who have not had a recorded risk assessment in the last 90 days.

```
SQL
SELECT
    p.SSN AS Patient_SSN,
    u.First_Name,
    u.Last_Name,
    ra.Risk_Level,
    MAX(ra.Assessment_Date) AS LastAssessmentDate
FROM PATIENT p
JOIN USER u ON p.SSN = u.SSN
JOIN RISK_ASSESSMENT ra ON p.SSN = ra.Patient_SSN
WHERE ra.Risk_Level = 'High'
GROUP BY p.SSN, u.First_Name, u.Last_Name, ra.Risk_Level
HAVING MAX(ra.Assessment_Date) < DATE_SUB(CURDATE(), INTERVAL 90 DAY)
ORDER BY LastAssessmentDate ASC;
```

5.3 Query 3: Facility Workload Analysis

Summarizes appointment volumes per facility along with active doctor and patient resource allocation.

```
SQL
SELECT
    f.Facility_Name,
    COUNT(DISTINCT a.Appointment_Number) AS TotalAppointments,
    COUNT(DISTINCT a.Patient_SSN) AS UniquePatients,
    COUNT(DISTINCT a.Doctor_SSN) AS ActiveDoctors
FROM FACILITY f
JOIN DEPARTMENT d ON f.Facility_Name = d.Facility_Name
JOIN APPOINTMENT a ON d.Department_Name = a.Department_Name
GROUP BY f.Facility_Name
ORDER BY TotalAppointments DESC;
```

5.4 Query 4: Doctor Weekly Schedule

Generates a consolidated weekly schedule for doctors showing upcoming appointments.

```
SQL
SELECT
    d.SSN AS Doctor_SSN,
    CONCAT(u.First_Name, ' ', u.Last_Name) AS DoctorName,
    a.Department_Name,
    a.Room_Number,
    a.Appointment_Date,
    CONCAT(pu.First_Name, ' ', pu.Last_Name) AS PatientName
FROM DOCTOR d
JOIN USER u ON d.SSN = u.SSN
JOIN APPOINTMENT a ON d.SSN = a.Doctor_SSN
JOIN PATIENT p ON a.Patient_SSN = p.SSN
JOIN USER pu ON p.SSN = pu.SSN
WHERE a.Appointment_Date BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 7
DAY)
ORDER BY d.SSN, a.Appointment_Date ASC;
```

5.5 Query 5: Medicated vs Non-Medicated Lab Analysis

Compares mean lab test values between patients with active prescriptions versus those without active prescriptions.

```sql
SELECT
    lt.Test_Name,
    CASE
        WHEN ap.Appointment_Number IS NOT NULL THEN 'On Medication'
        ELSE 'No Active Medication'
    END AS PatientStatus,
    ROUND(AVG(lt.Test_Result), 2) AS AverageLabResult,
    COUNT(DISTINCT lt.Accession_Number) AS TestCount
FROM LAB_TEST lt
JOIN APPOINTMENT a ON lt.Appointment_Number = a.Appointment_Number
LEFT JOIN APPOINTMENT_PRESCRIPTION ap
    ON a.Appointment_Number = ap.Appointment_Number
    AND ap.Status = 'Active'
GROUP BY
    lt.Test_Name,
    CASE
        WHEN ap.Appointment_Number IS NOT NULL THEN 'On Medication'
        ELSE 'No Active Medication'
    END
ORDER BY lt.Test_Name, PatientStatus;
```

---

6. Conclusion & Team Responsibilities

6.1 Project Summary

The Patient Risk Assessment System effectively addresses critical gaps in chronic disease monitoring by providing a fully normalized 25-table relational model capable of tracking patient risk levels, clinical appointments, prescriptions, lab test thresholds, and disease risk scoring models over time. The application of analytical queries supports clinical decision-making and resource optimization, directly furthering the goals of UN SDG 3.

6.2 Team Responsibilities Matrix

Team Member	Primary Contributions & Responsibilities	Sign-off / Verification
Navjot Singh Shienh (30332849)	Project Management, Requirements Gathering, Database Implementation	Complete

Team Member	Primary Contributions & Responsibilities	Sign-off / Verification
Iesrat Jahan Popy (30304908)	Conceptual Model, Logical Model, Data Population	Complete
Nikita Williams (30143759)	Logical Model, Normalization	Complete
Md Jubayer Khan (30336019)	Documentation, Database Testing	Complete